Real-Time Voting App

Node.js · Socket.IO · Bootstrap 5 | Live WebSocket Voting

Scrum 4 – Team 52

CIS 266 – Web Services | Instructor: Huda Judeh
Team 52 is Zach Kahler, Nova Denton-Perry, Bruce Schulz

Project Goals

What we set out to build

Core Objectives

- Allow multiple users to vote on a preferred free-time activity simultaneously
- Display live vote results to all connected users in real time
- Prevent duplicate votes per session
- Allow an admin to reset all votes at any time

Activities

- 🎮 Video Games
- 📖 Reading
- 🪢 Crafty Hobbies
- 💪 Physical Activity
- 🎲 Board Games

Key Feature

- ✓ Real-time updates via Socket.IO
- ✓ Multi-tab simultaneous voting
- ✓ Admin reset functionality

Methods / Technologies Used

Stack and tools powering the app

Technologies

- **Node.js**
  JavaScript runtime for the server
- **Socket.IO**
  WebSocket library for real-time, bi-directional communication
- **HTML / CSS**
  Front-end structure and layout
- **Bootstrap 5**
  UI styling and responsive components
- **WebSockets (ws://)**
  Protocol for persistent client-server connection

Architecture / How It Works

3-tier real-time event-driven flow

CLIENT
`app.js / index.html`

⟺

SOCKET.IO
`ws:// layer`

⟺

SERVER
`server/index.js`

- **On connect:** Server emits current vote totals to the new socket
- **On 'vote' event:** Server increments the correct counter; io.emit broadcasts to all
- **On 'reset' event:** Server zeroes all counters; io.emit clears all clients
- **Client 'update' handler:** Re-renders emoji counters live — no page reload

server > JS index.js > ...

```
1  const http = require('http').createServer();
2  const io = require('socket.io')(http, {
3    cors: {origin: '*'}
4  });
5
6  let videoGameVotes = 0;
7  let readingVotes = 0;
8  let craftyVotes = 0;
9  let physicalActivityVotes = 0;
10 let boardGameVotes = 0;
11
```

Code – Server-Side Voting Logic

server/index.js — vote event handler

How It Works

`socket.on('vote', ...)`
Fired when any client emits a vote event

`switch (voteValue)`
Routes the vote to the correct counter variable

`videoGameVotes++`
Increments the in-memory counter for that activity

`io.emit('update', {...})`
Broadcasts ALL current totals to every connected client

`Invalid selection`
Falls to default: logs the bad value to the terminal for debugging

server > JS index.js > ...

```
12  io.on('connection', (socket) => {
13    // send current vote counts to newly connected user
14    socket.emit('update', {videoGameVotes, readingVotes, craftyVotes, physicalActivityVotes, boardGameVotes});
15
16    socket.on('vote', (voteValue) => {
17      // update vote count
18      switch (voteValue) {
19        case 'videoGames':
20          videoGameVotes++;
21          break;
22        case 'reading':
23          readingVotes++;
24          break;
25        case 'crafty':
26          craftyVotes++;
27          break;
28        case 'physicalActivity':
29          physicalActivityVotes++;
30          break;
31        case 'boardGames':
32          boardGameVotes++;
33          break;
34        default:
35          console.log('invalid vote selection');
36      }
37
38      // broadcast updated totals
39      io.emit('update', {videoGameVotes, readingVotes, craftyVotes, physicalActivityVotes, boardGameVotes});
40    });
41
```

Code – Client-Side (Receiving Updates)

app/app.js — socket.on('update', ...) handler

How It Works

`socket.on('update', ...)`
Listens for the broadcast emitted by the server

`Destructured payload`
Extracts each vote count from the incoming object

`emoji.repeat(n)`
Renders n emoji characters to visually represent the count

`document.querySelector`
Finds the result span for each activity and updates it

`Zero page refresh`
DOM updates happen live — Socket.IO pushes the data

app > JS app.js > ...

```
4
5   // if user already voted before refreshing, disable the form immediately
6   if (voteSubmitted) {
7     document.querySelector('#voteStatus').innerText = 'Your vote has been submitted';
8     document.querySelector('#submit').classList.remove('btn-success');
9     document.querySelector('#submit').classList.add('btn-secondary');
10    document.querySelectorAll('input[name="activity"]').forEach(el => el.disabled = true);
11  }
12
```

Code – HTML Front-End

index.html — radio form and results display

Key HTML Elements

`<input type="radio">`
One radio input per activity, grouped by name="activity"

`value="videoGames"`
Matches the server-side switch case key exactly

`<button id="voteBtn">`
Submit Vote — disabled after click to prevent duplicates

`<span id="videoGameResults">`
Empty on load; filled with emojis by app.js on update

`<button id="resetBtn">`
Emits reset event; admin use only

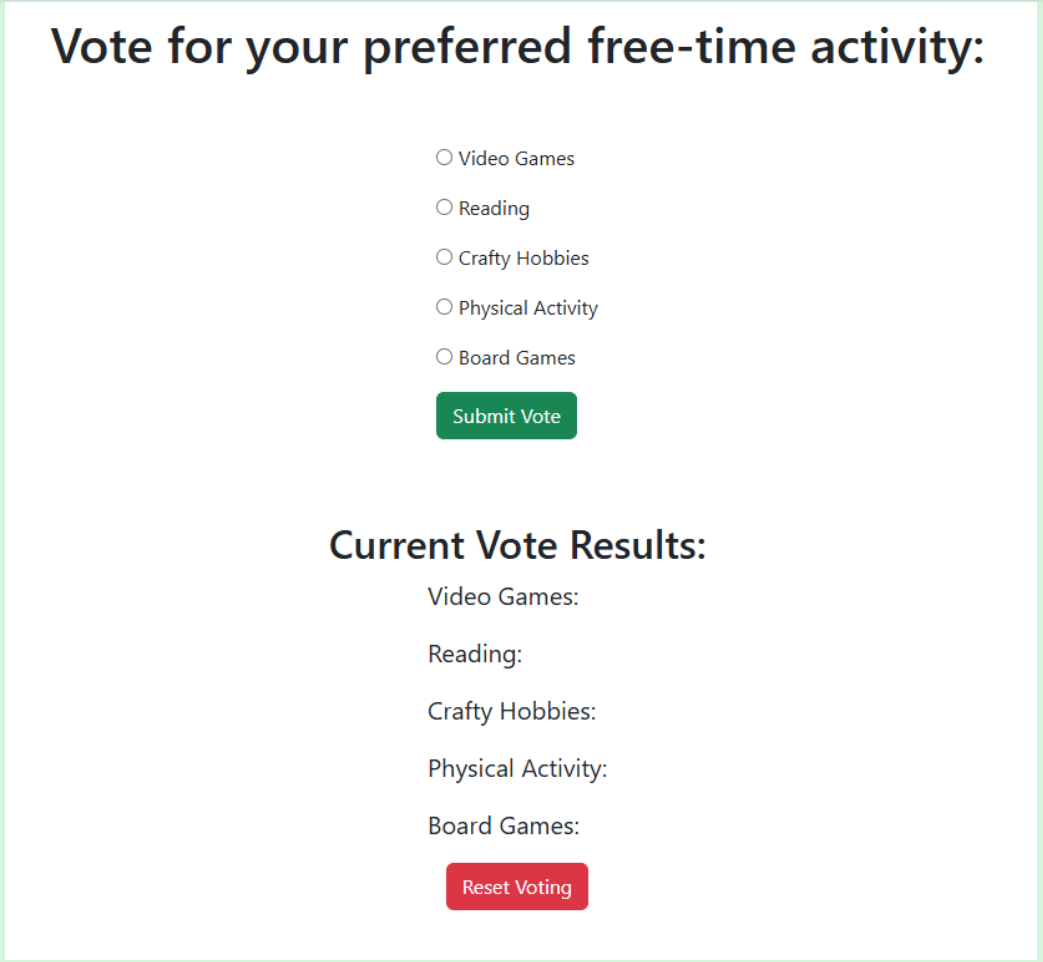
app > index.html > html > body.d-flex.flex-column.justify-content-center.align-items-center

```
2   <html lang="en">
11  <body class="d-flex flex-column justify-content-center align-items-center">
13
14    <form class="mt-5">
15      <p>
16        <input type="radio" id="videoGames" name="activity" value="videoGames">
17        <label for="videoGames">Video Games</label>
18      </p>
19
20      <p>
21      <input type="radio" id="reading" name="activity" value="reading">
22      <label for="reading">Reading</label>
23      </p>
24
25      <p>
26        <input type="radio" id="crafty" name="activity" value="crafty">
27        <label for="crafty">Crafty Hobbies</label>
28      </p>
29
30      <p>
31        <input type="radio" id="physicalActivity" name="activity" value="physicalActivity">
32        <label for="physicalActivity">Physical Activity</label>
33      </p>
34
35      <p>
36        <input type="radio" id="boardGames" name="activity" value="boardGames">
37        <label for="boardGames">Board Games</label>
38      </p>
39
40      <button id="submit" class="btn btn-rounded btn-success">Submit Vote</button>
41    </form>
42
43    <p id="voteStatus"></p>
44
45    <h2 class="mt-5">Current Vote Results:</h2>
46    <div id="results" class="fs-5">
47      <p>Video Games: <span id="videoGameResults"></span></p>
48      <p>Reading: <span id="readingResults"></span></p>
49      <p>Crafty Hobbies: <span id="craftyResults"></span></p>
50      <p>Physical Activity: <span id="physicalActivityResults"></span></p>
51      <p>Board Games: <span id="boardGameResults"></span></p>
52    </div>
53
54    <button id="reset" class="btn btn-rounded btn-danger">Reset Voting</button>
55
```

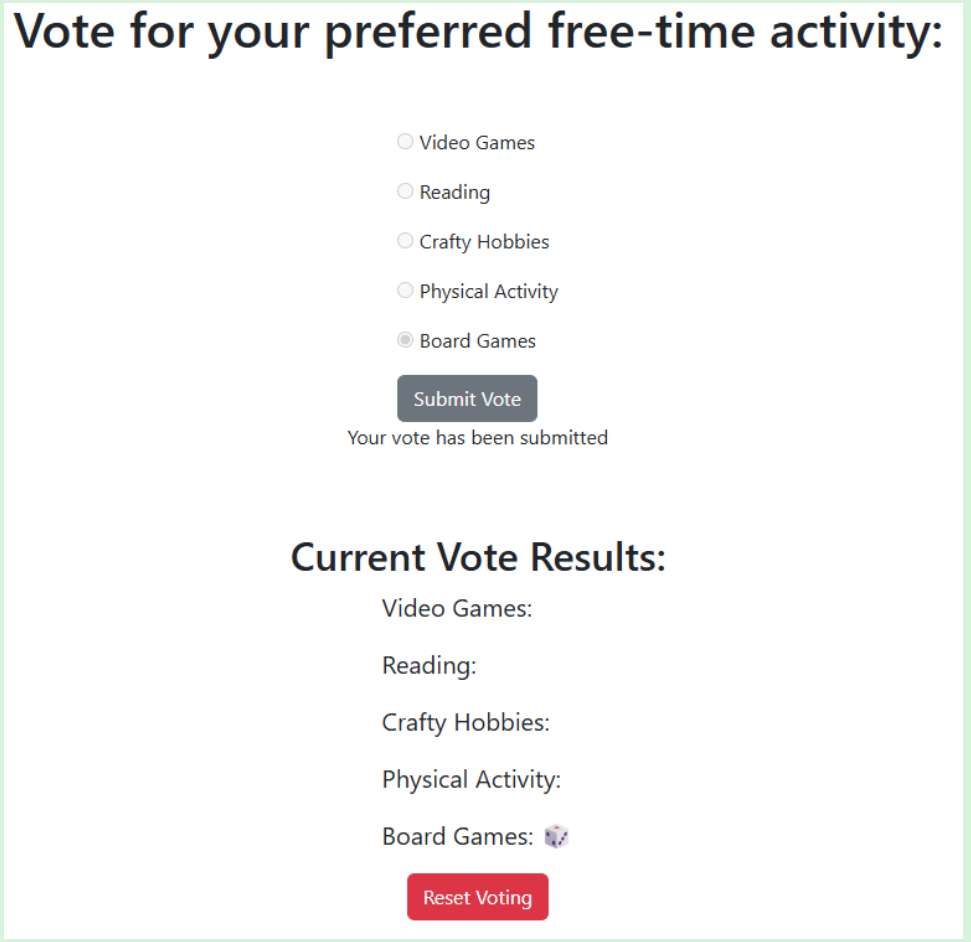
IO Captures – App Screenshots: Part 1

Running app in 4 states

1. Fresh App — all votes at 0



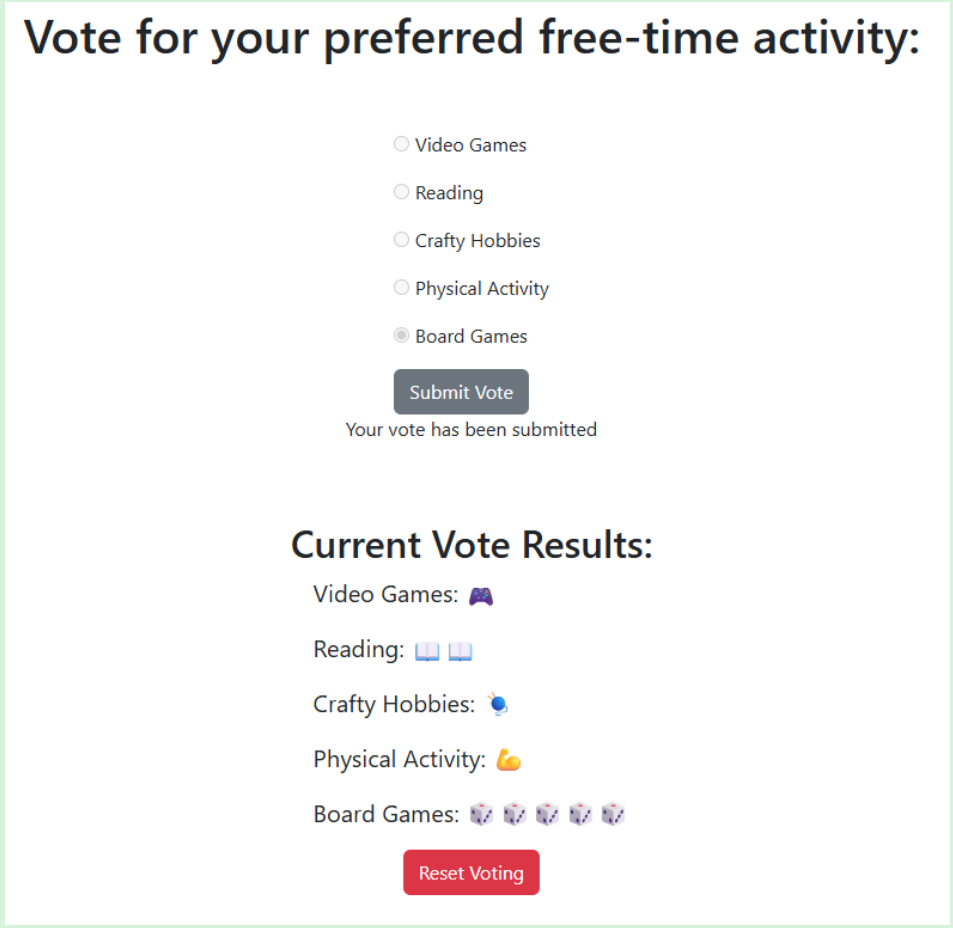
2. After Voting — submission confirmed



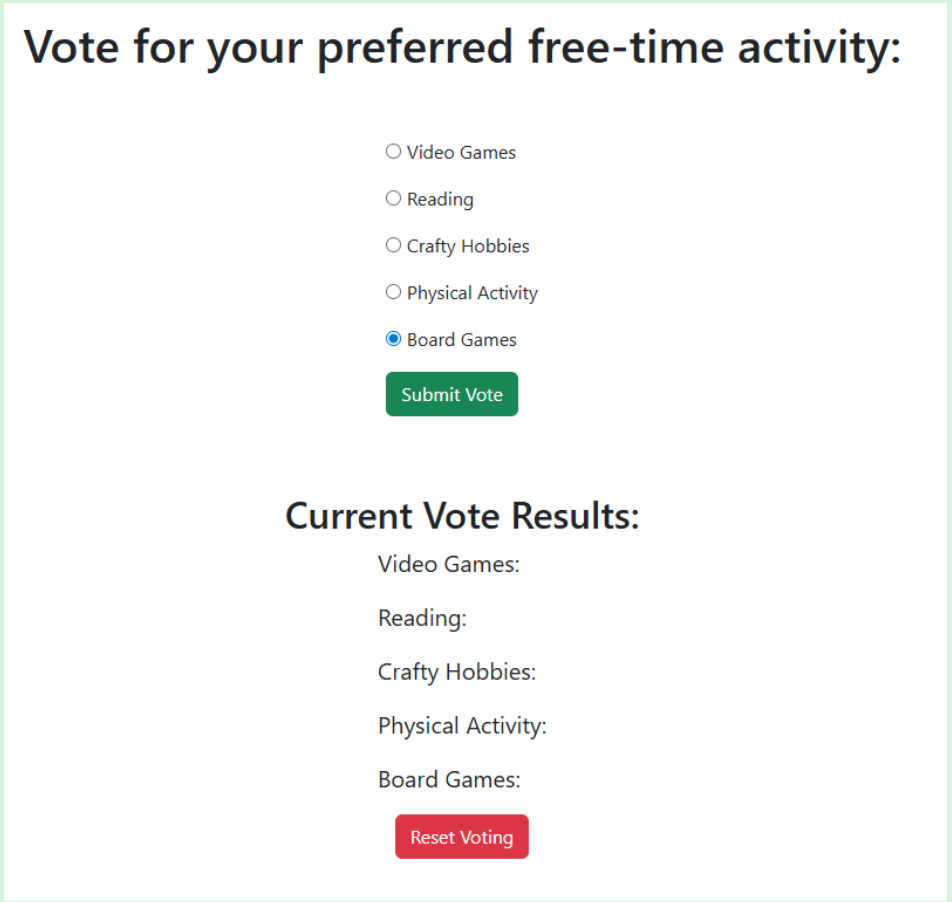
IO Captures – App Screenshots: Part 2

Running app in 4 states

3. Multiple Voters — emoji counters filling



4. After Reset — all emojis cleared



Testing & Debugging

How to verify and trace the app

How to Test

- Open multiple browser tabs to simulate voters
- Submit a vote in each tab — all tabs should update simultaneously
- Try voting twice in the same tab — button should be disabled
- Click Reset and confirm all tabs clear immediately

server > JS index.js > ...

```
12   io.on('connection', (socket) => {
16     socket.on('vote', (voteValue) => {
32           boardGameVotes++;
33           break;
34         default:
35           console.log('invalid vote selection');
36       }
37
```

How to Debug

- DevTools → Console: Socket.IO connection errors
- DevTools → Network → WS: inspect vote and update frames live
- Terminal: console.log fires on invalid vote values
- Add console.log(voteValue) in switch to trace incoming votes

Summary & Conclusion

What we built and learned

What We Built

✓ Real-time multi-user voting app using Socket.IO

✓ WebSocket push updates — no page refresh required

✓ Event-driven architecture: vote → broadcast → render

✓ Server-side state management in Node.js

✓ Duplicate-vote prevention and admin reset

Potential Improvements

→ Persist votes to a database (MongoDB / MySQL)

→ Add authentication — secure the reset endpoint

→ Display results as a live animated bar chart

→ Deploy to a cloud host (Render, Railway, Vercel)

→ Add a countdown timer before voting closes